--Chain of Thought Prompting
1) Automation
Context-As a Automation Architect generate me playwright framework structure
Instruction
-ASK me 5 questions one by one for framework creation
-Ask me the next question after me providing the answer
-Framework should be Enterprise following the industry standard
Step by step implementation

==========/
Answer
==========

Context-As a Automation Architect generate me playwright framework structure
Instruction
-ASK me 5 questions one by one for framework creation
-Ask me the next question after me providing the answer
-Framework should be Enterprise following the industry standard
Step by step implementation

Great — let's design an enterprise-grade Playwright automation framework step by step.

Question 1/5
Which programming language do you want to use with Playwright?

Options (choose one or specify your own):

TypeScript (most recommended for enterprise)

JavaScript

Python

Java (.NET/C# also possible but less common with Playwright)

☞ Please answer, then I'll move to the next question.

Python

Good choice — Python + Playwright is powerful and flexible for enterprise automation.

Question 2/5
Which test design pattern / architecture style do you want to follow?

Options (you can choose one or combine):

Page Object Model (POM)

Playwright Test style (fixtures-based, less POM heavy)

Hybrid (POM + reusable utilities + service layer) ✓ recommended for enterprise

Screenplay Pattern (advanced, scalable)

☞ Please choose your preferred approach.

Hybrid (POM + reusable utilities + service layer)

Excellent — Hybrid (POM + Utilities + Service Layer) is the right enterprise choice 👆

Question 3/5
What types of testing scope should the framework support?

You can choose multiple:

UI Testing (Web)

API Testing (REST/GraphQL)

Database Testing

Mobile Web / Responsive Testing

End-to-End (E2E) flows across microservices

☞ Example answer: UI + API + E2E

Tell me your selection, and I'll move to the next question.

UI Testing (Web) + E2E

Great — UI + E2E is perfect for a scalable enterprise setup 👆

Question 4/5
Which test runner & reporting strategy do you prefer?

Options (you can combine):

Pytest (highly recommended for Python ecosystem) ✅

Unittest (basic, less flexible)

Reporting tools:

Allure Reports (enterprise standard) ✅

HTML Reports (Pytest default / plugins)

Custom reporting + dashboards

☞ Example answer: Pytest + Allure

Please share your preference, and I'll ask the final question.

Custom reporting + dashboards

Good — custom reporting + dashboards is a strong enterprise move 👍 (especially for leadership visibility and trend analysis)

Question 5/5
What CI/CD, execution, and environment strategy do you want?

Please specify:

CI/CD tool:

Jenkins

GitHub Actions

GitLab CI

Azure DevOps

Execution:

Local

Dockerized execution

Kubernetes / scalable grid

Parallel execution needed? (Yes/No)

Environment handling:

Multiple envs (DEV/QA/UAT/PROD)?

Config management approach (env files, config service, secrets manager)

☞ Example answer:
Jenkins + Docker + Parallel execution + DEV/QA/UAT + env-based config

Once you answer this, I'll generate:

✅ Full enterprise Playwright (Python) framework structure
✅ Folder structure + design patterns
✅ Sample code (POM, service layer, utilities)
✅ Step-by-step implementation guide
✅ Review & improvement suggestions

GitHub Actions as CI/CD
Execution: Parallel execution
Environments: DEV,QA,UAT,PROD
env-based config

Perfect — now I have everything needed. I'll design a complete enterprise-grade Playwright (Python) hybrid framework tailored to your inputs.

✅ 1. Finalized Architecture Overview
Tech Stack

Language: Python

Framework: Playwright

Test Runner: Pytest

Design Pattern: Hybrid (POM + Service Layer + Utilities)

Scope: UI + E2E

CI/CD: GitHub Actions

Execution: Parallel

Environments: DEV / QA / UAT / PROD (env-based config)

Reporting: Custom dashboards

✅ 2. Enterprise Framework Folder Structure

```
playwright-python-framework/
│
├── config/
│   ├── config.yaml
│   ├── dev.yaml
│   ├── qa.yaml
│   ├── uat.yaml
│   ├── prod.yaml
│   └── config_loader.py
│
├── tests/
│   ├── e2e/
│   │   ├── test_user_journey.py
│   │   └── test_checkout_flow.py
│   │
│   └── ui/
│       ├── test_login.py
│       └── test_dashboard.py
│
```

```
├── pages/                  # Page Object Model
│   ├── base_page.py
│   ├── login_page.py
│   └── dashboard_page.py
│
├── services/               # Business logic / workflows
│   ├── auth_service.py
│   └── user_service.py
│
├── locators/
│   ├── login_locators.py
│   └── dashboard_locators.py
│
├── utils/
│   ├── logger.py
│   ├── helpers.py
│   ├── wait_utils.py
│   └── data_generator.py
│
├── fixtures/
│   ├── browser_fixture.py
│   ├── page_fixture.py
│   └── env_fixture.py
│
├── test_data/
│   ├── test_data.json
│   └── test_data.yaml
│
├── reports/
│   ├── raw/
│   └── dashboard/
│
├── hooks/
│   └── pytest_hooks.py
│
├── .github/workflows/
│   └── ci.yml
│
├── requirements.txt
├── pytest.ini
└── README.md
```

✅ 3. Key Design Concepts

🔷 Hybrid Framework Layers

Tests Layer

Only test logic

No UI locators or business logic

Pages (POM)

UI interactions

Element handling

Service Layer

Business workflows (login, checkout, etc.)

Reusable across tests

Utilities

Logging, waits, data, helpers

Fixtures

Browser setup, environment setup

✅ 4. Sample Implementation

🔽 Config Loader (Environment Handling)

```
# config/config_loader.py
import yaml
import os

def load_config():
    env = os.getenv("ENV", "qa")
    with open(f"config/{env}.yaml") as f:
        return yaml.safe_load(f)
```

🔽 Base Page

```
# pages/base_page.py
class BasePage:
    def __init__(self, page):
        self.page = page

    def click(self, locator):
        self.page.locator(locator).click()

    def fill(self, locator, value):
        self.page.locator(locator).fill(value)

    def get_text(self, locator):
        return self.page.locator(locator).inner_text()
```

🔽 Login Page

```
# pages/login_page.py
from pages.base_page import BasePage
from locators.login_locators import LoginLocators

class LoginPage(BasePage):

    def login(self, username, password):
        self.fill(LoginLocators.USERNAME, username)
```

```
        self.fill(LoginLocators.PASSWORD, password)
        self.click(LoginLocators.LOGIN_BTN)
```

🔹 Service Layer

```
# services/auth_service.py
from pages.login_page import LoginPage

class AuthService:

    def __init__(self, page):
        self.login_page = LoginPage(page)

    def login_user(self, username, password):
        self.login_page.login(username, password)
```

🔹 Pytest Fixture

```
# fixtures/page_fixture.py
import pytest
from playwright.sync_api import sync_playwright

@pytest.fixture(scope="function")
def page():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        context = browser.new_context()
        page = context.new_page()
        yield page
        browser.close()
```

🔹 Test Example (E2E)

```
# tests/e2e/test_user_journey.py
from services.auth_service import AuthService

def test_login_flow(page):
    auth = AuthService(page)
    auth.login_user("testuser", "password")
```

✅ 5. Parallel Execution Setup

```
pip install pytest-xdist
```

Run tests:

```
pytest -n auto
```

✅ 6. GitHub Actions (CI/CD)

```
# .github/workflows/ci.yml
name: Playwright Tests

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
```

```
    matrix:
      env: [dev, qa, uat]

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Install Python
      uses: actions/setup-python@v4
      with:
        python-version: 3.10

    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        playwright install

    - name: Run tests
      run: |
        pytest -n auto
      env:
        ENV: ${{ matrix.env }}
```

✅ 7. Custom Reporting Strategy (Enterprise Level)

Instead of Allure:

Suggested Approach:

Store results in JSON

Push to:

ElasticSearch / MongoDB

Build dashboard using:

Grafana / Kibana / Power BI

Example:

```
# hooks/pytest_hooks.py
import json

def pytest_runtest_logreport(report):
    if report.when == "call":
        result = {
            "test": report.nodeid,
            "status": report.outcome,
        }
        with open("reports/raw/results.json", "a") as f:
            f.write(json.dumps(result) + "\n")
```

✅ 8. Enterprise Enhancements (MUST HAVE)

✔ Retry mechanism (flaky test handling)

✓ Screenshot & video capture on failure
✓ Smart waits (avoid hard waits)
✓ Tagging (smoke/regression)
✓ Test data externalization
✓ Logging (structured logs)
✓ Secrets handling (GitHub secrets)

✅ 9. Step-by-Step Implementation Plan
Step 1
Setup repo + install Playwright + Pytest

Step 2
Create base structure (folders)

Step 3
Implement config management

Step 4
Build BasePage + utilities

Step 5
Develop POM layer

Step 6
Add service layer

Step 7
Write test cases

Step 8
Enable parallel execution

Step 9
Integrate GitHub Actions

Step 10
Build custom reporting dashboard

✅ 10. Architecture Review (Expert Feedback)
👍 Strengths
Scalable (microservices-ready)

Clean separation of concerns

CI/CD ready

Parallel execution optimized

Enterprise reporting capability

⚠ Improvements (Next Level)
Add API layer integration (future-ready)

Introduce contract testing

Add test orchestration layer

Implement AI-based flaky test detection

Integrate Docker for consistency

If you want next level 🚀
I can help you with:

Full working GitHub repo template

Docker setup

Advanced dashboard (Grafana)

Converting this into interview-ready architecture explanation

==========
2) Testing Example
Generate Testplan , take 5 questions step by step once after answering proceed with next question
==========
Context-As a Test Lead generate me a test plan document
Instruction
-ASK me 5 questions one by one for creation
-Ask me the next question after me providing the answer
-Test plan should be the industry standard
Step by step implementation
==========

Answer:
==========

Context-As a Test Lead generate me a test plan document
Instruction
-Ask me 5 questions one by one for creation
-Ask me the next question after me providing the answer
-Test plan should be the industry standard
Step by step implementation

Great — we'll build a solid industry-standard Test Plan document step by step.

Question 1 of 5: Project Overview
What is the type of application and domain for which you need the test plan?

☞ Please provide details like:

Domain (e.g., Banking, Insurance, E-commerce, Healthcare, etc.)

Application type (Web, Mobile, API, Microservices, etc.)

High-level purpose of the application

Once you answer, I'll move to the next question.

Domain: Banking
Application: Web

Good — Banking + Web gives us a solid foundation.

Question 2 of 5: Scope of Testing
What types of testing should be included in this test plan?

☞ You can choose from (or add your own):

Functional Testing

Integration Testing

System Testing

Regression Testing

API Testing

Performance Testing

Security Testing

UAT Support

Accessibility Testing

Also mention if there are any out-of-scope items.

Once you answer, I'll proceed to the next question.

No out of scope items

Got it — but I still need one important detail before proceeding.

You mentioned no out-of-scope items, which is fine 👍
However, I need to know which testing types to include in the plan.

Question 2 (continued): Scope of Testing
Please confirm the testing types to be included.

☞ For example, you can say:

“Include all standard testing types”
OR

Provide a list like:

Functional

API

Regression

Performance

Security

etc.

Once you confirm, I’ll move to Question 3.

Include all standard testing types

Perfect — that gives us a comprehensive scope, which is ideal for a Banking application ✅

Question 3 of 5: Test Environment & Data
Please provide details about:

Environments involved
(e.g., DEV, QA, UAT, PROD)

Test Data Strategy
Will you use production-like data, synthetic data, or masked data?

Any dependencies on external systems (e.g., payment gateways, third-party APIs)?

☞ Example answer:
DEV, QA, UAT, PROD
Synthetic + masked data, external dependencies exist

Once you answer, I’ll proceed to Question 4.

DEV, QA, UAT, PROD + Masked data , no external dependencies

Great — that’s clear and well-structured 👍

Question 4 of 5: Tools & Automation Strategy
Please provide details on:

Test Management & Defect Tracking Tools
(e.g., JIRA, Azure DevOps, TestRail)

Automation Tools / Framework
UI Automation (e.g., Selenium, Playwright)

API Automation (e.g., RestAssured, Postman, Insomnia)

CI/CD Tools
(e.g., Jenkins, GitHub Actions, Azure Pipelines)

Reporting & Logging
(e.g., Extent Reports, Allure, custom dashboards)

Once you share this, I'll move to the final question (Question 5).

JIRA
UI Automation: Selenium
API Automation: Insomnia
GitHub Actions: CI/CD

Good — nice and practical toolset 👍

Question 5 of 5: Execution Strategy & Team Structure
Please provide:

Execution Strategy
Parallel or Sequential execution?

Frequency (e.g., daily, per build, sprint-wise)

Team Structure
Roles involved (e.g., QA Lead, QA Engineers, Automation Engineers)

Onshore/Offshore (if applicable)

Release Strategy
Agile (Scrum), Waterfall, Hybrid?

👉 Example:
Parallel execution, daily CI runs
Team: QA Lead + 4 QA Engineers + 2 Automation Engineers
Agile Scrum (2-week sprint)

Once you answer this, I'll generate a complete industry-standard Test Plan document for you 🚀

Execution Strategy
Parallel
Release Strategy: Agile (Scrum)
Roles: QA Lead, QA Engineers, Automation Engineers, QA Manager

Excellent — you've provided all the key inputs. I'll now generate a complete industry-standard Test Plan document tailored for your Banking Web application.

📄 TEST PLAN DOCUMENT
1. Introduction
This Test Plan defines the strategy, scope, resources, and schedule for testing the Banking Web Application. It ensures the application meets business, functional, performance, and security requirements before production release.

2. Objectives
Validate end-to-end banking workflows

Ensure high reliability, security, and performance

Detect defects early and reduce production risk

Support continuous delivery in agile environment

3. Scope of Testing
In Scope
All standard testing types are included:

Functional Testing

Integration Testing

System Testing

Regression Testing

API Testing

Performance Testing

Security Testing

User Acceptance Testing (UAT) Support

Accessibility Testing

Out of Scope
None (Full coverage approach)

4. Test Strategy
4.1 Testing Approach

A hybrid testing approach combining manual and automation will be followed:

Manual testing for exploratory, UAT, and edge cases

Automation for regression, API, and repeatable scenarios

4.2 Levels of Testing
Unit testing (by Dev team)

Integration Testing

System Testing

UAT

5. Test Environment

Environment	Purpose
DEV	Initial testing by developers
QA	Functional, API, regression testing
UAT	Business validation
PROD	Production validation (smoke checks)

6. Test Data Management
Masked Data will be used to ensure data privacy and compliance

No external system dependencies

Test data will be version-controlled where applicable

7. Tools & Technologies
Test Management & Defect Tracking
JIRA

Automation Tools
UI Automation: Selenium

API Testing: Insomnia

CI/CD
GitHub Actions

Reporting
CI-based execution reports

Custom dashboards (can be enhanced later)

8. Test Execution Strategy
Execution Type: Parallel execution

Automation Runs: Triggered via CI/CD pipelines

Frequency:

Per commit (smoke tests)

Daily (regression suite)

Sprint end (full regression + performance)

9. Entry & Exit Criteria
Entry Criteria
Requirements are finalized and approved

Test environment is ready

Test data is available

Build is stable

Exit Criteria
All critical test cases executed

No open critical/high defects

Test coverage meets agreed threshold

Stakeholder sign-off received

10. Defect Management
All defects will be logged in JIRA

Severity & Priority defined as:

Critical

High

Medium

Low

Daily triage meetings will be conducted

11. Roles & Responsibilities
Role Responsibility
QA Manager Overall QA governance & strategy
QA Lead Test planning, coordination, reporting
QA Engineers Test case design & execution
Automation Engineers Framework & script development
12. Risks & Mitigation
Risk Mitigation
Environment instability Use dedicated QA environment
Data issues Maintain masked & controlled datasets

Tight deadlines Prioritize critical test scenarios
Automation delays Parallel manual execution
13. Deliverables
Test Plan Document

Test Cases & Test Data

Automation Scripts

Test Execution Reports

Defect Reports

Test Summary Report

14. Schedule (High-Level)
Aligned with Agile sprints:

Test planning: Sprint start

Execution: Continuous during sprint

Regression: End of sprint

UAT: Pre-release phase

15. Approval
QA Lead

QA Manager

Project Stakeholders

✅ Step-by-Step Implementation Plan (Practical Execution)
Step 1: Setup
Configure JIRA project

Setup Selenium framework

Integrate with GitHub Actions

Step 2: Test Design
Write test scenarios & cases

Prepare masked test data

Step 3: Automation Development
Develop UI automation scripts

Configure API collections in Insomnia

Step 4: CI/CD Integration
Setup pipelines in GitHub Actions

Enable parallel execution

Step 5: Test Execution
Run smoke tests per build

Execute regression suites daily

Step 6: Reporting
Generate reports via CI

Track defects in JIRA

Step 7: Release Support
Support UAT

Provide sign-off reports

If you want, I can next:

Review this like a QA Manager (audit feedback)

Convert this into a Word/PDF document

Or tailor it for interview / leadership presentation 🚀

Home Assignment:04.04.2026
==========
1. Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)
==========
Context-As a Test Lead generate me a test strategy document for the Domain Insurance
Instruction
-ASK me 6 questions one by one for creation
-Ask me the next question after me providing the answer
-Test Strategy should be the industry standard
Step by step implementation
==========

Answer

Context-As a Test Lead generate me a test strategy document for the Domain Insurance
Instruction
-ASK me 6 questions one by one for creation
-Ask me the next question after me providing the answer
-Test Strategy should be the industry standard
Step by step implementation

Great—let’s build this step by step like a real industry-grade Test Strategy 👆

Question 1 of 6
Which type of insurance domain are we targeting?

For example:

Life Insurance

Health Insurance

Motor Insurance

Property & Casualty

Reinsurance

Or a combination (end-to-end insurance platform)

👉 Please specify the domain and a brief description of the system (if available).

Life Insurance

Got it — Life Insurance domain 👆

Question 2 of 6
What is the scope of the application/system under test?

For example, which modules are included:

Policy Administration (create, update, renew, surrender)

Underwriting

Claims Management

Billing & Payments

Agent/Broker Portal

Customer Portal

Integrations (banks, third-party services, KYC, etc.)

👉 Please list the modules/features that are in scope for testing.

Underwriting

Perfect — Underwriting module 👆

Question 3 of 6
What is the type of application and architecture?

For example:

Web application / Mobile app / API-based / Microservices

Monolith or Microservices architecture

Cloud-based (AWS/Azure/GCP) or On-prem

Any third-party integrations involved?

☞ Please provide details about the application type and architecture.

Web application and Microservices architecture

Great — Web application with Microservices architecture 👆

Question 4 of 6
What are the key testing types you want to include in the strategy?

For example:

Functional Testing

API Testing

Integration Testing

System Testing

Regression Testing

Performance Testing

Security Testing

UAT Support

☞ Please specify which testing types should be covered (you can say “all standard types” if you prefer full coverage).

Functional Testing
API Testing
Integration Testing
System Testing
Regression Testing

Performance Testing
Security Testing
UAT Support

Excellent — that's a comprehensive coverage 👍

Question 5 of 6
What is the test environment setup?

For example:

Number of environments (DEV, QA, UAT, PROD-like, etc.)

Availability of dedicated QA environment

Test data strategy (masked production data / synthetic data)

Any environment constraints or dependencies?

☞ Please provide details about the test environments and data setup.

DEV, QA, UAT, PROD and Synthetic data , no dependecies

Great — that's clear 👍

Question 6 of 6 (Final)
What are the tools and technologies you prefer or currently use?

For example:

Test Management: JIRA, TestRail, ALM

Automation: Selenium, Playwright, Cypress

API Testing: Postman, RestAssured

Performance: JMeter, LoadRunner

Security: OWASP ZAP, Burp Suite

CI/CD: Jenkins, GitHub Actions, Azure DevOps

☞ Please list your preferred tools (or say "suggest best tools" if you want me to recommend).

JIRA, Selenium, insomnia, LoadRunner,Burp Suite, Jenkins

Perfect — now I have all the inputs 👍
Below is your industry-standard Test Strategy Document for a Life Insurance – Underwriting Module (Web + Microservices).

📄 Test Strategy Document
Project: Life Insurance – Underwriting Module
Application Type: Web Application (Microservices Architecture)
Prepared By: Test Lead

1. Introduction
This Test Strategy defines the overall approach for testing the Underwriting module in a Life Insurance system. It ensures quality, reliability, performance, and security of underwriting workflows including risk assessment, policy approval, and integration with downstream systems.

2. Scope of Testing
✅ In Scope
Underwriting workflow validation

Risk assessment rules and decision engine

Data validation and business rules

API interactions between microservices

Integration with external systems (if any simulated)

UI validation for underwriting screens

❌ Out of Scope
Other insurance modules (Claims, Billing, etc.)

Production validation (except smoke checks)

3. Test Objectives
Ensure underwriting decisions are accurate and compliant

Validate end-to-end workflows across microservices

Identify defects early and reduce production leakage

Ensure system scalability and performance under load

Validate security vulnerabilities and data protection

4. Test Approach
4.1 Functional Testing
Validate business rules for underwriting

Boundary value analysis and equivalence partitioning

Negative and positive scenarios

4.2 API Testing
Validate REST APIs using Insomnia

Verify request/response, status codes, contracts

Mock dependent services if required

4.3 Integration Testing
Validate communication between microservices

Data consistency across services

Error handling and retry mechanisms

4.4 System Testing
End-to-end underwriting journey

UI + Backend validation

Cross-browser testing using Selenium

4.5 Regression Testing
Automated regression suite using Selenium

Executed in CI/CD pipeline via Jenkins

Focus on critical underwriting flows

4.6 Performance Testing
Load and stress testing using LoadRunner

Validate system under peak load conditions

Identify bottlenecks and response time issues

4.7 Security Testing
Vulnerability scanning using Burp Suite

OWASP Top 10 validation

Authentication, authorization, and data encryption checks

4.8 UAT Support
Support business users during UAT phase

Defect triage and quick turnaround

Sign-off validation

5. Test Environment

Environment	Purpose
DEV	Developer testing
QA	Functional, integration, system testing

UAT Business validation
PROD Smoke validation post-deployment
Test Data Strategy
Synthetic test data will be used

Data sets will cover all underwriting scenarios

Data refresh strategy defined per cycle

6. Test Tools
Category Tools
Test Management JIRA
UI Automation Selenium
API Testing Insomnia
Performance LoadRunner
SecurityBurp Suite
CI/CD Jenkins
7. Test Deliverables
Test Strategy Document

Test Plan

Test Cases & Test Data

Automation Scripts

Daily/Weekly Status Reports

Defect Reports

Test Summary Report

8. Entry & Exit Criteria
Entry Criteria
Requirements are signed off

Test environment is ready

Test data is available

Exit Criteria
All critical test cases executed

No open high/critical defects

Test coverage achieved (≥ 95%)

Stakeholder sign-off

9. Defect Management
Defects tracked in JIRA

Severity & priority classification

Daily triage meetings

Root cause analysis for critical defects

10. Risk & Mitigation
Risk Mitigation
Environment instability Dedicated QA environment
Data issues Synthetic data with full coverage
Integration failures Mock services and stubs
Tight timelines Automation and parallel execution
11. Automation Strategy
Selenium for UI automation

API automation (extendable using frameworks)

Integrated with Jenkins for CI/CD

Nightly regression runs

Focus on high business impact scenarios

12. Reporting & Metrics
Test execution progress

Defect density

Test coverage

Pass/Fail rate

Automation coverage

13. Governance & Communication
Daily stand-ups

Weekly status reports

Defect triage calls

Stakeholder reviews

✓ Step-by-Step Implementation Plan
Requirement analysis & test strategy approval

Test planning & estimation

Test case design & review

Test environment setup

Test data preparation

Test execution (Functional → Integration → System)

Automation development in parallel

Regression & performance testing

Security testing

UAT support & sign-off

Test closure & reporting

If you want, I can next:

Convert this into a resume-ready document

Or tailor it for interview explanation (Test Lead level)

Or add real-time examples/scenarios from underwriting 👆

2. Design the automation framework structure using a chain-of-thought approach

Output: Analyze and review the generated results
==========

Context-As a Test Automation Lead generate me the automation framework structure Instruction
- ASK me 5 questions one by one for creation
- Ask me the next question after me providing the answer
- Automation structure should be the industry standard
- Analyze and review the generated result

Step by step implementation
==========

Context-As a Test Automation Lead generate me the automation framework structure Instruction
- ASK me 5 questions one by one for creation
- Ask me the next question after me providing the answer
- Automation structure should be the industry standard
- Analyze and review the generated result

Step by step implementation

Great — we'll build a solid industry-standard automation framework step by step.

Question 1:
What programming language and automation tool do you want to use?

☞ Example:

Java + Selenium

Python + PyTest / Playwright

JavaScript + Cypress / Playwright

Please provide your choice, and I'll ask the next question.

Java + Selenium

Great choice — Java + Selenium is widely used and very stable for enterprise frameworks.

Question 2:
Which test framework do you want to use with Selenium?

☞ Common options:

TestNG (most popular for enterprise frameworks)

JUnit

Cucumber (BDD) (if you prefer Gherkin/feature files)

You can also combine (e.g., TestNG + Cucumber).

Please share your choice, and I'll move to the next question.

Cucumber (BDD)

Nice — Cucumber (BDD) is excellent for collaboration and readable test scenarios 👆

Question 3:
What type of application under test are you automating?

☞ Options (you can choose multiple):

Web application (UI)

API testing

Mobile application

Hybrid (UI + API)

Also, mention if it follows:

Monolith / Microservices architecture

Once you answer, I'll move to the next question.

Hybrid (UI + API) and Microservices architecture

Excellent — Hybrid + Microservices is a strong real-world setup 👍

Question 4:
What build tool and dependency management do you prefer?

👉 Common options:

Maven (most widely used with Java + Selenium)

Gradle

Also, let me know:

Do you want integration with CI/CD tools like Jenkins / GitHub Actions?

Once you answer, I'll ask the final question.

Maven and Use Jenkins for CI/CD

Perfect — Maven + Jenkins is a strong enterprise combination 👍

Question 5 (Final):
What reporting and logging tools do you want in your framework?

👉 Common choices:

Reporting: Extent Reports / Allure Reports / Cucumber Reports

Logging: Log4j / SLF4J

You can choose multiple.

Once you answer this, I'll:

Generate a complete industry-standard framework structure

Provide step-by-step implementation

Do a review & improvements analysis

Extent Reports, logging you should consider as per the current industry

Great — I have all the inputs. I'll design a modern, enterprise-grade automation framework based on your choices and include current industry practices.

✅ Final Framework Summary (Your Inputs)
Language: Java

UI Automation: Selenium

BDD: Cucumber

API: REST-based (Microservices)

Build Tool: Maven

CI/CD: Jenkins

Reporting: Extent Reports

Logging: SLF4J + Logback (industry standard)

🏭 Industry-Standard Framework Architecture
🔷 High-Level Design Pattern
BDD (Cucumber)

Page Object Model (POM)

Service Layer (for API)

Factory Design Pattern (Driver management)

Dependency Injection (PicoContainer / Spring optional)

📁 Project Structure (Recommended)
```
automation-framework/
│
├── src/main/java/
│   ├── config/
│   │   ├── ConfigReader.java
│   │   ├── ConfigLoader.java
│   │
│   ├── driver/
│   │   ├── DriverFactory.java
│   │   ├── DriverManager.java
│   │
│   ├── pages/
│   │   ├── BasePage.java
│   │   ├── LoginPage.java
│   │
│   ├── api/
│   │   ├── ApiClient.java
│   │   ├── RequestBuilder.java
```

```
│   │   ├── ResponseValidator.java
│   │
│   ├── utils/
│   │   ├── WaitUtils.java
│   │   ├── ScreenshotUtils.java
│   │   ├── JsonUtils.java
│   │
│   ├── constants/
│   │   ├── FrameworkConstants.java
│
├── src/test/java/
│   ├── stepdefinitions/
│   │   ├── LoginSteps.java
│   │   ├── ApiSteps.java
│   │
│   ├── runners/
│   │   ├── TestRunner.java
│   │
│   ├── hooks/
│   │   ├── Hooks.java
│   │
│   ├── testdata/
│   │   ├── testdata.json
│
├── src/test/resources/
│   ├── features/
│   │   ├── login.feature
│   │   ├── api.feature
│   │
│   ├── config/
│   │   ├── config.properties
│
├── reports/
├── logs/
├── pom.xml
```

⚙ Key Framework Components

1. Driver Management

Thread-safe WebDriver using ThreadLocal

Supports parallel execution

2. BDD Layer (Cucumber)

Feature files → Business readable

Step Definitions → Glue code

3. Page Object Model (UI)

Keeps locators + actions separated

Reusable components

4. API Layer
Centralized API client (use RestAssured)

Reusable request/response handling

5. Hooks
Before/After scenarios

Screenshot on failure

Driver initialization & teardown

6. Reporting
Extent Reports integration with Cucumber

Attach screenshots on failure

7. Logging (Modern Standard)
SLF4J + Logback

Log levels: INFO, DEBUG, ERROR

Separate log file per execution

🚀 Step-by-Step Implementation
Step 1: Create Maven Project
Add dependencies in pom.xml:

Selenium

Cucumber (Java + JUnit/TestNG runner)

RestAssured

Extent Reports

SLF4J + Logback

Step 2: Setup WebDriver Factory
Create DriverFactory.java

Use ThreadLocal for parallel runs

Step 3: Configure Cucumber
Add feature files

Create TestRunner.java

Configure glue paths

Step 4: Implement Page Object Model

BasePage → common methods

Page classes → UI interactions

Step 5: Build API Layer
Create reusable API client

Handle headers, auth, payload

Step 6: Add Hooks
Initialize driver before scenario

Capture screenshot after failure

Step 7: Integrate Reporting
Configure Extent Reports adapter

Attach logs + screenshots

Step 8: Logging Setup
Add logback.xml

Configure file + console logging

Step 9: Jenkins Integration
Create pipeline job in Jenkins

Execute:

mvn clean test
Publish reports

Step 10: Parallel Execution
Enable in Cucumber/TestNG

Configure thread count in Maven

🔍 Review & Analysis (Industry Benchmark)
✅ Strengths
✓ Scalable for microservices
✓ Supports UI + API (Hybrid)
✓ BDD improves business collaboration
✓ CI/CD ready
✓ Parallel execution enabled
✓ Clean separation of concerns

⚠️ Improvements (Recommended Enhancements)
1. Add Design Patterns
Singleton (Driver)

Builder Pattern (API requests)

2. Add Test Data Strategy
Externalize data (JSON/Excel/DB)

Use data-driven approach

3. Add Environment Handling
DEV / QA / UAT configs

Use profiles in Maven

4. Add Retry Mechanism
Retry failed tests automatically

5. Add Docker Support
Run tests in containers

6. Add Cloud Execution
Integrate with:

Selenium Grid

BrowserStack / LambdaTest

✭ Final Verdict
This framework is:

✓ Enterprise-ready

✓ Scalable

✓ Maintainable

✓ Industry-aligned (2026 standards)